Laboratório 04 — as quatro buscas, medidas

D2 — Fundamentos de Agentes Inteligentes · Encontro 3, bloco 2 Pacote: `D2-04-busca-nao-informada.zip` · 25 minutos

O que você já tem, e o que falta

Você já tem	O que falta
A formulação do lab-03, testada pelo teste de sanidade	Apontar o esqueleto para ELA — uma constante no topo
O motor `comum/busca.py`, com as seis buscas provadas	Rodar as quatro buscas sobre a SUA instância maior
O esqueleto JÁ RODA: aponta para o catálogo, imprime a tabela em Markdown	A conclusão escrita: alguma busca achou caminho mais barato que a largura?
O gabarito dos números do catálogo: `MEDICOES.md` do motor, com o professor	A tabela guardada — é o item 4 da entrega final (25% da nota)

No caminho padrão, **o código já vem pronto**. O seu trabalho é rodar, entender e explicar por escrito o que a tabela mostrou — a rubrica pontua a análise, não a autoria do código.

Os dois caminhos

- **Padrão:** troque a constante `FORMULACAO` no topo do `esqueleto.py` para a sua formulação (`"meu_problema.py:MEU_PROBLEMA"`), execute, leia a tabela, escreva a conclusão.
- **Desafio:** a fronteira e o conjunto de explorados são seus — preencha as duas funções marcadas com TODO no esqueleto (`minha_busca_em_largura` e `meu_custo_uniforme`), a partir do protocolo do slide 46 e **sem abrir** `comum/busca.py`. Depois troque `USAR_IMPLEMENTACAO_PROPRIA` para `True` e compare as suas linhas com as do motor: mesmo custo, mesmo número de ações.

Trocar de caminho é livre, nos dois sentidos e sem justificar — inclusive durante o laboratório.

Os passos

1. Extraia o `D2-04` na raiz do repositório e aponte a constante `FORMULACAO` do `esqueleto.py` para a sua formulação do lab-03.
2. Rode as quatro buscas sobre a sua instância maior:

```
python esqueleto.py
```

1. Preencha a tabela na entrega: custo, ações, expandidos, gerados, segundos — a saída já sai em Markdown, pronta para colar.

2. Responda: alguma busca achou caminho mais barato que a largura? A resposta depende do seu subproblema — em alguns é sim (custo uniforme vence), em outros não (os custos são parecidos), e as duas respostas rendem análise.
3. Guarde a tabela: ela é o item 4 da entrega final. O lab-05 acrescenta as linhas da heurística a esta mesma tabela.
4. Desafio: com as suas buscas prontas, rode o `teste.py` do pacote — `python -m pytest teste.py -q`. A família "desafio" compara as suas buscas com o motor nos três subproblemas do catálogo e na sua formulação.

Critério de aceite

Tabela preenchida com as quatro buscas e uma conclusão escrita — o mesmo critério do slide 21, literal.

Quanto tempo isto deve levar

Vinte e cinco minutos. O esqueleto já roda no catálogo de fábrica; apontar para a sua formulação e rodar leva dois. O resto do tempo é leitura da tabela e escrita da conclusão — que pode terminar em casa, desde que os números saiam agora.

Se travar

1. **Não terminou em dez segundos?** O espaço está grande demais — reduza a instância (menos itens, menos verificações) antes de depurar qualquer outra coisa.
2. **Custo uniforme pior que a largura em custo?** É defeito de formulação, não resultado — impossível com formulação correta. Quase sempre é custo negativo ou transição que não é função; o teste de sanidade do lab-03 aponta qual das duas.
3. **A tabela mostra um número esquisito de nós da profundidade limitada (sempre minúsculo)?** É a ressalva de contagem, abaixo. Não é defeito — é medição com definição diferente, e a análise precisa dizer isso.

A ressalva de contagem (slide 20) — registre na sua análise

`nos_expandidos` conta cada nó retirado da fronteira, repetidos inclusive; `nos_gerados`, cada nó criado. As duas primeiras buscas têm fronteira; a **profundidade limitada é recursiva e não tem fronteira** — nela, `nos_expandidos` conta entradas na recursão, sem repetidos. Parte da vantagem aparente dela na tabela é diferença de contagem, não eficiência. Reconhecer o limite da própria medição vale nota.
